

RAPPORT DE LAB

Data Warehousing

LAB 2 — Data Warehousing with SSIS

Chargement de l'entrepôt, Data Flow, SQL Task et Slowly Changing Dimensions

Nom : FATIHI Yasser

Module : Data Warehousing

Outils utilisés : SQL Server, SSMS, Visual Studio / SSDT, SSIS, SSAS, SSRS

Remarque : les emplacements de captures doivent être remplacés par vos screenshots réels avant dépôt.

Introduction

Ce lab poursuit le travail du Lab 1 : la base sample_dw créée précédemment est utilisée comme entrepôt cible. L'objectif est de charger les dimensions et la table de faits avec SSIS, puis d'implémenter une Slowly Changing Dimension pour conserver l'historique des clients.

- Charger Customers, Products, Dates et InternetSales / FactSales.
- Comparer le chargement par requête SQL, SQL Task et Data Flow SSIS.
- Mettre en place un SCD Type 1 / Type 2 pour la dimension Customers.
- Historiser les modifications et les suppressions au lieu de perdre l'historique.

Partie A — Loading data into a data warehouse

A.1 — Chargement de Customers avec SSMS

La première méthode consiste à exécuter une requête INSERT INTO depuis SSMS afin de charger la dimension Customers à partir d'AdventureWorksDW.

```
USE sample_dw;
INSERT INTO dbo.Customers
(CustomerAlternateKey, FullName, EmailAddress, BirthDate, MaritalStatus,
Gender, Education, Occupation, City, StateProvince, CountryRegion)
SELECT C.CustomerAlternateKey,
       COALESCE(C.FirstName + ' ', '') + COALESCE(C.LastName, '') AS FullName,
       C.EmailAddress, C.BirthDate, C.MaritalStatus, C.Gender,
       C.EnglishEducation, C.EnglishOccupation,
       G.City, G.StateProvinceName, G.EnglishCountryRegionName
FROM AdventureWorksDW2017.dbo.DimCustomer AS C
INNER JOIN AdventureWorksDW2017.dbo.DimGeography AS G
ON C.GeographyKey = G.GeographyKey;
```

Capture d'écran : Requête INSERT Customers exécutée dans SSMS

EMPLACEMENT POUR VRAIE CAPTURE — à remplacer par le screenshot pris dans SSMS / Visual Studio / SSDT.

Question : Why can we consider the Customers table a denormalized table?

Réponse : The Customers table is denormalized because it stores customer information and geographic information in the same table. In AdventureWorksDW, these attributes are split between DimCustomer and DimGeography. In sample_dw, City, StateProvince and CountryRegion are stored directly in Customers, which reduces joins and makes reporting easier.

A.2 — Chargement avec SQL Task dans SSIS

Dans le package load_by_SSIS.dtsx, un Execute SQL Task nommé SQLTaskLoadCustomers exécute la même logique de chargement dans un pipeline SSIS.

Capture d'écran : SQLTaskLoadCustomers configuré dans le Control Flow

EMPLACEMENT POUR VRAIE CAPTURE — à remplacer par le screenshot pris dans SSMS / Visual Studio / SSDT.

A.3 — Chargement de Products avec Data Flow

La deuxième méthode utilise un Data Flow : une source OLE DB extrait les données de DimProduct, DimProductSubcategory et DimProductCategory, puis une destination OLE DB charge la table Products.

```
SELECT P.ProductAlternateKey AS ProductKey,  
       P.EnglishProductName AS ProductName,  
       P.Color, P.Size,  
       S.EnglishProductSubcategoryName AS SubcategoryName,  
       C.EnglishProductCategoryName AS CategoryName  
FROM AdventureWorksDW2017.dbo.DimProduct AS P  
INNER JOIN AdventureWorksDW2017.dbo.DimProductSubcategory AS S  
ON P.ProductSubcategoryKey = S.ProductSubcategoryKey  
INNER JOIN AdventureWorksDW2017.dbo.DimProductCategory AS C  
ON S.ProductCategoryKey = C.ProductCategoryKey;
```

Capture d'écran : Data Flow ExtractProduct vers LoadProduct

EMPLACEMENT POUR VRAIE CAPTURE — à remplacer par le screenshot pris dans SSMS / Visual Studio / SSDT.

Question : As opposed to using the SQL task, the data flow includes a source adapter and a destination adapter. What are the advantages of using the data flow?

Réponse : The Data Flow is more flexible and easier to monitor than a single SQL Task. It separates extraction and loading, provides column mappings, supports transformations such as Sort, Lookup and Data Conversion, and makes errors easier to redirect and debug. It also gives a visual ETL pipeline that is easier to maintain.

A.4 — Chargement de Dates et InternetSales / FactSales

Le même principe est appliqué pour charger la dimension Dates puis la table de faits. Les clés des dimensions doivent être disponibles avant le chargement de la table de faits.

```
-- Source for Dates
SELECT DateKey, FullDateAlternateKey AS FullDate,
       SUBSTRING(CONVERT(NVARCHAR(8), FullDateAlternateKey, 112), 5, 2)
       + ' ' + EnglishMonthName AS MonthNumberName,
       CalendarQuarter, CalendarYear
FROM AdventureWorksDW2017.dbo.DimDate;

-- Source for Fact table / InternetSales
SELECT C.CustomerDwKey, P.ProductKey, F.OrderDateKey,
       F.OrderQuantity, F.SalesAmount, F.UnitPrice, F.DiscountAmount
FROM AdventureWorksDW2017.dbo.FactInternetSales AS F
INNER JOIN AdventureWorksDW2017.dbo.DimCustomer AS Cust
ON F.CustomerKey = Cust.CustomerKey
INNER JOIN sample_dw.dbo.Customers AS C
ON Cust.CustomerAlternateKey = C.CustomerAlternateKey
INNER JOIN sample_dw.dbo.Products AS P
ON F.ProductKey = P.ProductKey;
```

Capture d'écran : Sequence Container : Customers, Products, Dates puis InternetSales

EMPLACEMENT POUR VRAIE CAPTURE — à remplacer par le screenshot pris dans SSMS / Visual Studio / SSDT.

Question : Explain why the sequence of loading data should follow the order in the figure above?

Réponse : The dimensions must be loaded before the fact table because the fact table contains foreign keys that reference Customers, Products and Dates. If the fact table is loaded first, referential integrity errors can occur. Loading dimensions first also allows the ETL to resolve surrogate keys before inserting fact rows.

Capture d'écran : Exécution réussie du package de chargement

EMPLACEMENT POUR VRAIE CAPTURE — à remplacer par le screenshot pris dans SSMS / Visual Studio / SSDT.

Partie B — Slowly Changing Dimension

B.1 — Préparation de Customers pour l'historisation

Pour suivre les versions historiques des clients, deux colonnes de validité sont ajoutées à la table Customers.

```
ALTER TABLE sample_dw.dbo.Customers
ADD ValidFrom DATE,
    ValidTo DATE;
```

Capture d'écran : Colonnes ValidFrom et ValidTo ajoutées dans Customers

EMPLACEMENT POUR VRAIE CAPTURE — à remplacer par le screenshot pris dans SSMS / Visual Studio / SSDT.

B.2 — Source staging CustomerInformation

La table stg.CustomerInformation représente la source opérationnelle utilisée pour tester les changements. Elle est créée et alimentée avec le script create_stagingtable_2024.sql.

Capture d'écran : Table stg.CustomerInformation créée et alimentée

EMPLACEMENT POUR VRAIE CAPTURE — à remplacer par le screenshot pris dans SSMS / Visual Studio / SSDT.

B.3 — Configuration du Slowly Changing Dimension Wizard

- Business key : CustomerAlternateKey.
- Fixed attributes : attributs qui ne doivent pas changer, par exemple BirthDate ou Gender.
- Type 1 : attributs mis à jour directement, par exemple EmailAddress.
- Type 2 : attributs historisés, par exemple MaritalStatus ou CountryRegion.
- ValidFrom et ValidTo identifient les versions courantes et expirées.

Capture d'écran : SCD Wizard : mapping des colonnes et Business Key

EMPLACEMENT POUR VRAIE CAPTURE — à remplacer par le screenshot pris dans SSMS / Visual Studio / SSDT.

Capture d'écran : SCD Wizard : Type 1 / Type 2 et ValidFrom / ValidTo

EMPLACEMENT POUR VRAIE CAPTURE — à remplacer par le screenshot pris dans SSMS / Visual Studio / SSDT.

Capture d'écran : Package SCD_Customers exécuté avec succès

EMPLACEMENT POUR VRAIE CAPTURE — à remplacer par le screenshot pris dans SSMS / Visual Studio / SSDT.

Question : Do you see the difference in execution between the insertion of a new row and the update of an existing row?

Réponse : Yes. When a new customer appears in the source, the SCD component inserts it as a new row. When an existing customer changes, Type 1 attributes overwrite the current value, while Type 2 attributes close the old version by setting ValidTo and insert a new current version with a new validity period.

B.4 — Historisation des suppressions

Pour les clients supprimés de la source staging, le Data Flow compare les clients actifs de l'entrepôt avec stg.CustomerInformation. Les lignes non trouvées sont redirigées vers un OLE DB Command qui les marque comme non courantes.

```
SELECT CustomerAlternateKey, FullName, EmailAddress, BirthDate,
       MaritalStatus, Gender, Education, Occupation,
       City, StateProvince, CountryRegion, Age
FROM sample_dw.dbo.Customers
WHERE CurrentFlag = 1;

UPDATE sample_dw.dbo.Customers
SET CurrentFlag = 0,
    ValidTo = GETDATE()
WHERE CustomerAlternateKey = ?;
```

Capture d'écran : Data Flow pour historiser les clients supprimés

EMPLACEMENT POUR VRAIE CAPTURE — à remplacer par le screenshot pris dans SSMS / Visual Studio / SSDT.

Capture d'écran : Résultat SSMS : clients avec CurrentFlag = 0 et ValidTo renseigné

EMPLACEMENT POUR VRAIE CAPTURE — à remplacer par le screenshot pris dans SSMS / Visual Studio / SSDT.

Question : How was your Customers Dimension table impacted?

Réponse : The rows that no longer exist in the source were not physically deleted from the warehouse. They were marked as inactive by setting CurrentFlag = 0 and ValidTo = the execution date. This preserves the history while indicating that these customers are no longer current.

Question : What do you have to take into consideration when using this dimension table later on for reporting?

Réponse : For current-state reports, we must filter on CurrentFlag = 1 or ValidTo IS NULL to avoid using expired versions. For historical reports, the date of the fact must be compared with ValidFrom and ValidTo so that each fact is linked to the correct version of the customer.

Conclusion

Dans ce lab, nous avons chargé l'entrepôt sample_dw avec plusieurs méthodes SSIS, organisé le pipeline avec un Sequence Container, puis implémenté une Slowly Changing Dimension pour conserver les modifications et suppressions de la dimension Customers. Cette approche rend l'entrepôt plus fiable pour les analyses BI et prépare les données pour l'OLAP dans le Lab 3.